

Experiment Design for Computer Sciences (0AL0400)

Paper Study – "Benchmarking in Optimization: Best Practice and Open Issues" Part I

Claus Aranha

caranha@cs.tsukuba.ac.jp

University of Tsukuba, Department of Computer Sciences

Version 2024.1 (Updated June 14, 2024)

Motivation

- Recent paper describing the best practices in CS experiments involving benchmark analysis;
- Covers several of the topics of this course: A good review of the topics;
- Important for you too: Optimization-like problems occurs in many CS disciplines;

Download the paper: <https://arxiv.org/abs/2007.03488>

Overview of the Lecture

Today

- Defining the Goals of a Benchmarking Experiment;
- Choosing problems and algorithms;
- How to measure the result of the experiment;

Next Week

- Types of Analysis;
- Experimental Design;
- Presentation of the Results;

What is an optimization algorithm?

Consider a function $\mathbb{R}^N \rightarrow \mathbb{R}$. An optimization algorithm finds the input $X \in \mathbb{R}^N$ that minimizes the output.

- **Applications:** Design of products; Parametrization of Models; System Control; Machine Learning;
- **Classes of Algorithms:** Numerical; Heuristic; Meta-heuristic (search-based); Exhaustive search;
- **Classes of Problems:** Constrained; Multiple Solutions; Non-continuous; Mixed-Integer; Multiple Objectives;

Benchmark Experiments

There is an interest to develop more efficient optimization algorithms. To examine and evaluate new algorithms, optimization researchers often use Benchmark Experiments.

Benchmarks are arbitrary mathematical functions, sometimes based on real world application problems, that are used to test the efficiency of new optimization algorithms.

The No Free Lunch Theorem (NFLT) – Wolpert and Macready, 1997

Optimization algorithms cannot be equally good in all problem classes:

- Limit analysis of algorithm suitability to the problem **instances** being tested;
- Research new problem classes, and match algorithms to specific classes;
- Be careful about generalizing experiment results to other problem instances;
- Be **VERY** careful about generalizing results to other problem **classes**;

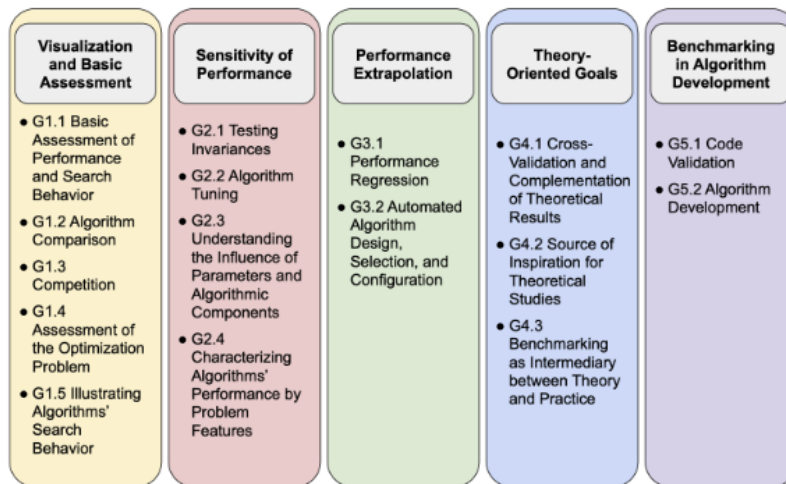
What is a Optimization Benchmarking Study?

A "benchmarking study" selects a set of optimization algorithms, a set of benchmark problems, and studies the results of applying the algorithms to the selected problems.

- Scientific Motivations: Better understanding of algorithms and problems;
- Engineering Motivations: Find solutions for very specific solutions;
- Advertising Motivations: Encourage the use of specific algorithms, problems or technologies;

Careful consideration of the study goals plays a **HUGE** role in the selection of algorithms, problems, performance measure, experiment design, analysis, conclusions, etc.

Common Goals of Benchmarking Studies



1- Visualization and Basic Assessment

- **1.1 - Basic Assessment of Performance and Behavior:** How does one specific Algorithm Configuration performs at one specific Problem Instance? Considerations for Stochasticity and robustness. Considerations for evaluation measures;
- **1.2 - Algorithm Comparison:** Most common in literature. Extension of the above for multiple Algorithms / Algorithm Configurations. Specify the most suitable algorithm configuration for a problem class or instance;
- **1.3 - Competition:** Compare several algorithms on specific set of problem instances, and specific performance measures; Can promote development of new ideas, but can also overfit/exaggerate the importance of certain problems/measures;
- **1.4 - Assessment of Problem:** Improve the understanding of a problem: How close is the benchmark function to reality? What is the class of the problem? What is the fitness landscape? How difficult is the problem?
- **1.5 - Illustrating the Search Behavior:** What is the robustness of the algorithm? How fast does it converge? What are the characteristics of its behavior?

2- Sensitivity of Performance

- **2.1 - Testing Invariances:** Testing if the performance of the algorithm is invariant to certain characteristics of the problem (eg: translation invariance)
- **2.2 - Algorithm Tuning:** Finding the most suitable configuration (parameters and hyper parameters) of an algorithm to a specific instance or class of problem;
- **2.3 - Understanding the influence of parameters:** How does the performance of the algorithm changes with the change of parameters. What parameters have higher or lower influence, depending on the problem instance? What is the specific effect of a parameter on the algorithm or problem instance? Compare with 2.2.
- **2.4 - Characterizing the algorithm by problem feature:** Related to 2.3, how does the characteristics of a problem class/instance affect the algorithm's performance? What characteristics have higher or lower effect on the algorithm?

3- Benchmark as Training: Performance Extrapolation

- **3.1 - Performance Regression:** Extrapolating the performance of the algorithm from problem classes and instances that were tested, to problems classes and instances that were not tested. How much **can we expect** the performance of the algorithm to change or degrade? How do we select problems to evaluate this degradation?
Related to ideas of train/testing, transfer learning, and zero shot in Machine Learning;
- **3.2 - Automated Algorithm Design, Selection and Configuration:** If we know the influence of parameter configuration and problem characteristics to an algorithm, it should be possible to automatically select appropriate parameters from the characteristics of an unseen problem instance. The goal of this kind study is to develop training data and guidelines to such a task, given an automated configuration framework.

4- Theory Oriented Goals

- **4.1 - Complementation of Theoretical Results:** Replication, Confirmation, and Extension of theoretical results for larger problems;
- **4.2 - Inspiration for Theoretical Studies:** Studies in a large quantity of problems or algorithms to observe patterns, and then try to explain those patterns through theoretical construction. The theory can explain, extend, or bound the experimental observations.
- **4.3 - Bridge between theory and practice:** Transform knowledge from theory into actual algorithms that are applicable to specific problems with specific issues;

5 - Benchmark in Algorithm Development

- **5.1 - Source Code Validation:** Verify that a program (instance of algorithm) performs as expected (theory or literature results). If the program does not behave as expected from past knowledge, this indicates the necessity of a code review, and the existence of bugs.
- **5.2 - Algorithm Development:** Benchmark studies can be used to identify weak spots in the literature, and to find the needs for new algorithms and methods. This includes initial empirical examinations of new ideas, fine-tuning future research directions.

What are all these objectives good for?

- Think about your scientific objective carefully: Most people automatically go for comparison studies, but for some scientific questions, other kinds of studies might be more appropriate;
- Think about your experiment design carefully: Depending on the kind of study, the way you design the experiment will change. Having a clear vision of the goal of the study is important. Also you can compare to similar studies (if you know what those studies are)

Short Exercise: How does this apply to you?

Consider experiments that you have performed so far (in this course, or in your master research). How would you classify the goal of that experiment? Would a different goal have been more useful to you? How do you transform some of these goals into your discipline?

Selecting Problems and Algorithms

After selecting the objective of the study, we need to choose the algorithms (algorithm configurations) under study, and the problem set (problem classes, problem instances);

Problem Class vs Problem Instance

- A **problem** / **problem class** specifies the general characteristics of the given optimization problem: Is it unimodal/multimodal? Is it continuous/discrete? What function represents the problem?

Example: The "Sphere" function;

- A **problem instance** specifies the implementation of one specific case of the problem class. It can include the values for constants in the problem formula, which can be changed to generate new instances. Or the number of dimensions or scale of the problem. **Different instances of the same problem class can have different difficulty!**

Example: The 5-dimensional sphere function $f : \mathbb{R}^5 \rightarrow \mathbb{R}, x \rightarrow \alpha \sum_{i=1}^d x_i^2 + \beta$, centered at the origin, with multiplicative parameter α and shift parameter β .

Problem Set

In general, the goal of a benchmark study include understanding the performance of algorithms not on one problem instance, but on a variety of problems instances and classes. Therefore, we need to always consider the **Problem Set** of an study:

Problem Set:

- Problem Class A
 - Problem A1
 - Problem Instance A1-a
 - Problem Instance A1-b
 - Problem Instance A1-c
 - Problem A2
 - Problem A3
- Problem Class B
- Problem Class C

Desirable Characteristics of a Benchmark Problem Set

- **Diverse:** Different difficulties, different classes, different characteristics;
- **Representative:** Correspond to difficulties and characteristics encountered in real world problems;
- **Scalable / Tunable:** By changing parameters, we can increase or decrease the difficulty, or generate variations of the same class of problems, as necessary.
- **Known Solutions / Performance:** We know some target values (absolute or relative) that the algorithm should reach if performing well;

Evaluating the Quality of a Problem Set

How can we evaluate the characteristics (diversity, representation, scalability) of a problem set?

Some past research approaches:

- Clustering of characteristics of problem instances;
- Clustering of performance of algorithms on the problem set;
- Automated creation and analysis of problem instances;
- Sensitivity analysis focused on **problem/problem instance** (How easy a problem gets as a parameter changes)
- Analysis of past results in the literature (are new algorithms obtaining significant changes in results?)

The result of this kind of research can provoke the creation of new benchmark sets to cover quality issues in old benchmark sets.

Generalization: Algorithm vs Algorithm Configuration

It is important to consider if our studies and experiments give us information about an algorithm, or about a specific instance of an algorithm:

- **Algorithm:** The CMA-ES algorithm;
- **Algorithm Configuration:** The pycma-es implementation, with population size 8, budget 100, restart strategy X, etc...

The performance of an algorithm can vary intensely depending on its implementation and configuration.

Generalization: Hyperparameter Handling

The performance of algorithm configurations depends on the hyperparameter settings and the problem characteristics. Because of this, it is necessary to tune all algorithms involved in a study.

"Literature Parameters": Even if an algorithm has been tuned in a different study from the literature, there are interaction effects between the algorithm configuration and the problem characteristics. These interaction effects require the re-tuning to guarantee an accurate evaluation of the performance of an algorithm (configuration) on a benchmark set. (Also, use parameter fine-tuning tools such as iRACE and SMAC)

Question

In what experimental situations the fine tuning of algorithms (i.e., the use of "literature parameters") would be sufficient? What should be considered in those situations?

Algorithm characteristics and the "State of the Art"

Many studies involve the comparison of several algorithms. The choice of algorithms should be influenced by several factors:

- **Problem features:** Different algorithms are appropriate for different problem classes and feature. A literature review is important for this choice, along with preliminary experiments to evaluate the properties of the algorithm on the problem.
- **State of the Art:** As technology develops, it is important to compare algorithms against the most recent version **and implementation** of classical algorithms. Searching for the state of the art from algorithm competitions is also recommended. Note that the state of the art may depend on the problem's characteristics;

Other factors regarding algorithm selection

- **Optimization Budget:** Optimization Budget is the amount of computation available for performing an optimization task (usually in terms of fitness evaluations). Different algorithms may perform better with low budgets (surrogate optimization algorithms) vs high budget (Evolution Strategy algorithms). This is an example of how problem characteristics affect the choice of state-of-the-art.
- **Initialization:** Optimization performance depends heavily on the initial conditions (starting close to the optimal may improve the results). Therefore all algorithms should have the same starting conditions, and a variety of starting conditions (seeds) should be examined to avoid "coincidental" results.

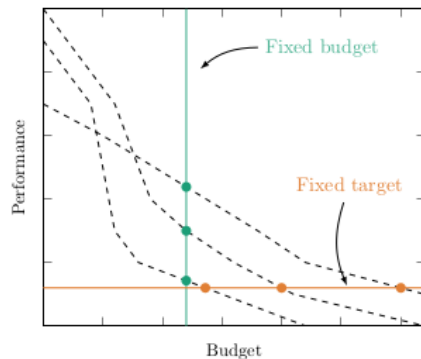
In general, the choice of competing algorithm should be analyzed carefully, and informed by the goal of the experiment.

Measuring Performance

Optimization Benchmark Experiments usually ask one of two questions:

- "How fast can the algorithms achieve a given solution quality?"
- "What solution quality can the algorithms achieve with a given budget?"

The most common way to visualize the result of an optimization algorithm is a graph with the computation budget on the x-axis, and the performance measure on the y-axis.



Measuring Budget

The budget of an algorithm can be measured in different ways:

- **Time:** Wall clock time, or CPU time; (Important when the optimization problem can be evaluated very quickly, eg: SAT)
- **Functions Evaluations:** How many times the optimization function was called (important when the optimization problem cannot be evaluated quickly, eg: simulations)

Measuring Solution Quality

- There are many **problem-specific** measures of solution quality;
 - Comparing quality across problems becomes an issue;
- Generalized solution quality: Distance (absolute or relative) to optimal solution;
 - Method for obtaining relative distance (normalization) can be an issue;
- Number of constraints violation;

Measuring Algorithm Robustness

Reasons for volatility (change) in repeated results:

- Stochastic behavior of algorithm (random search, random factors in algorithm);
- Initial condition of the optimization (seed, initial population);
- Noisy problem (stochasticity of the problem, change in data sets, physical conditions)

Measuring Robustness of algorithm performance:

- **With Fixed Budget:** Mean or median of the algorithm performance across multiple repetitions;
- **With Target performance:** Count of failed/successful runs; Expected Running Time (ERT) measure; Penalized Average Runtime (PAR/PQR) measure;
- **Quantiles, Standard Deviations, Confidence Intervals:** Smaller measures of variability in the results indicate higher robustness;

Summary

- **Selecting the goal of the experiment:** The goal of an algorithm experiment can be much more than "compare two algorithms".
- **Selecting algorithm and problem sets:** Algorithms and Problem Sets will affect each other, and should be chosen carefully. Difference between algorithm and algorithm instance, problem and problem instance;
- **Performance Measures:** There are many ways to measure the performance of an algorithm. Fixed budget vs Fixed target evaluation. Evaluation of Robustness.

Exam and Final Report

- **Exam:** I hope to give an example exam next class (maybe only on manaba). The topics are the contents of all lectures (including these two lectures). Calculations will not be necessary.
- **Final Report:** I will post the final report to Manaba ASAP. It will involve performing

About these Slides

These slides were made by Claus Aranha, 2022. You are welcome to copy, re-use and modify this material.

These slides are a modification of "Design and Analysis of Experiments (2018)" by Felipe Campelo, used with permission.

Individual images in some slides might have been made by other authors. Please see the following references for those cases.

Image Credits I

[Page 7] Table of Benchmarking Studies goal from the "best practices" paper

[Page 23] Budget Performance graph from "Best Practices" paper